

CHEAT SHEETS

A Complete Reference for Regular Expressions

- Metacharacters
- Character Classes
- Anchors
- Quantifiers
- Groups
- Lookarounds
- Substitution
- Practical Patterns
- Python · JS · Java · PHP · Go

9 REFERENCE SHEETS

CONTENTS

CORE SYNTAX	ADVANCED FEATURES	APPLIED
01 Foundations	05 Groups & References	08 Practical Patterns
02 Character Classes	06 Lookarounds	09 Language Syntax
03 Anchors & Boundaries	07 Substitution & Flags	
04 Quantifiers		

Sheet 1

Foundations

syntax · metacharacters · flags · escaping · testing

Sheet 2

Character Classes

[] · shorthands · negation · ranges · dot · unicode

Sheet 3

Anchors & Boundaries

^ · \$ · \b · \B · \A · \Z · multiline · word boundary

Sheet 4

Quantifiers

* · + · ? · {n,m} · greedy · lazy · possessive

Sheet 5

Groups & References

capturing · non-capturing · named groups · backreferences · alternation

Sheet 6

Lookarounds

lookahead · lookbehind · negative · zero-width · assertions

Sheet 7

Substitution & Flags

replace · sub · tokens · case modifiers · split · verbose

Sheet 8

Practical Patterns

email · URL · phone · date · IP · password · slug · hex color

Sheet 9

Language Syntax

Python · JavaScript · Java · PHP · Go · API comparison

WHAT IS A REGULAR EXPRESSION?

A **regular expression** (regex) is a sequence of characters that defines a search pattern. Engines scan a string left-to-right, attempting to match the pattern at each position. Regex is built into virtually every modern language and is used for *searching, validating, extracting, and replacing* text. The same core syntax works across Python, JavaScript, Java, Go, and PHP — only the API wrapper differs.

KEY FACTS

- › Regex engines are **greedy by default** — they match as much as possible.
- › Matching is **case-sensitive** by default; use the `i` flag to ignore case.
- › A pattern with no anchors can match *anywhere* in the string.
- › `/pattern/flags` is JS literal syntax; Python uses `r"pattern"` raw strings.
- › Always test patterns on real data — edge cases are common.

METACHARACTERS

CHAR	MEANING	EXAMPLE
.	Any character except newline	<code>a.c</code> → "abc", "alc"
^	Start of string (or line in multiline)	<code>^Hi</code> → "Hi there"
\$	End of string (or line in multiline)	<code>end\$</code> → "the end"
*	0 or more of previous token	<code>ab*</code> → "a", "ab", "abbb"
+	1 or more of previous token	<code>ab+</code> → "ab", "abbb"
?	0 or 1 of previous token (optional)	<code>colou?r</code> → "color", "colour"
{n,m}	Between n and m repetitions	<code>\d{2,4}</code> → "12", "1234"
	Alternation (OR)	<code>cat dog</code> → "cat" or "dog"
()	Capturing group	<code>(ab)+</code> → "ababab"
[]	Character class	<code>[aeiou]</code> → any vowel
\	Escape next character	<code>\.</code> → literal dot

NOTES

- `/pattern/flags` — JS regex literal; no quotes needed
- `r"..."` — Python raw string; backslashes are not processed by Python before regex sees them
- `test()` — returns boolean; faster than `match()` when you only need yes/no
- `match()` vs `search()` — Python `match()` anchors to start; `search()` scans entire string
- `findall()` — returns all non-overlapping matches as a list
- `fullmatch()` — entire string must satisfy the pattern

BASIC USAGE — JAVASCRIPT & PYTHON

```
// — JavaScript —
const str = "Hello World 2024";

/hello/i.test(str);           // true (case-insensitive)
str.match(/\d+/);             // ["2024"]
str.match(/\w+/g);            // ["Hello", "World", "2024"]
str.replace(/World/, "Regex"); // "Hello Regex 2024"
str.search(/\d/);              // 12 (index of first digit)
"a,b,c".split(/,/);           // ["a", "b", "c"]

// — Python —
import re
s = "Hello World 2024"

re.search(r'\d+', s).group()   # '2024'
re.findall(r'\w+', s)          # ['Hello', 'World', '2024']
re.sub(r'World', 'Regex', s)   # 'Hello Regex 2024'
re.split(r',+', 'a,b,c')       # ['a', 'b', 'c']
bool(re.fullmatch(r'\d{4}', '2024')) # True
```

FLAGS / MODIFIERS

FLAG	NAME	EFFECT
i	Case-insensitive	Treats A–Z same as a–z
g	Global (JS)	Find all matches, not just first
m	Multiline	<code>^</code> / <code>\$</code> match each line
s	Dotall / Single-line	<code>.</code> matches newline too
x	Extended / Verbose	Whitespace & comments ignored
u	Unicode (JS)	Full Unicode point matching
y	Sticky (JS)	Match only at lastIndex position

ESCAPING SPECIAL CHARACTERS

TO MATCH...	WRITE...	REASON
.	<code>\.</code>	Unescaped <code>.</code> matches any char
* + ?	<code>*</code> <code>\+</code> <code>\?</code>	Quantifier metacharacters
() [] { }	<code>\(</code> <code>\)</code> <code>\[</code> <code>\]</code> <code>\{</code> <code>\}</code>	Group / class / repetition delimiters
^ \$	<code>\^</code> <code>\\$</code>	Anchor metacharacters
\	<code>\ </code> <code>\\</code>	Alternation and escape itself
/ (JS)	<code>\/</code>	Closes a JS regex literal

COMMON MISTAKES

- › Forgetting to escape `.` — it matches any character, not just a literal dot.
- › Using Python `re.match()` when `re.search()` is needed — `match()` only checks the start of the string.
- › Omitting the `g` flag in JS `replace()` — only the first occurrence is replaced without it.
- › Writing `"\d"` (non-raw) in Python — the backslash is consumed by Python before the regex engine sees it.

TIPS

- › Use regex101.com or regexr.com to build and test patterns interactively with real-time explanations.
- › Always use raw strings `r"..."` in Python — it prevents double-escaping headaches.
- › Compile frequently-used patterns with `re.compile()` (Python) or store regex literals in `const` (JS) for performance.

SUMMARY

- › Regex defines a **pattern** matched against a string left-to-right.
- › Metacharacters (`.` `^` `$` `*` `+` `?` `|` `\` `(` `)` `[` `]` `{` `}`) have special meaning — escape them with `\` to match literally.
- › Flags (`i`, `g`, `m`, `s`) modify how the engine interprets the pattern.
- › API differences are shallow — the same pattern works across languages.

CHARACTER CLASSES

A **character class** (written inside `[...]`) matches *exactly one character* from the set you define. Ranges like `[a-z]` expand to all characters between those Unicode code points. **Negated classes** (`[^...]`) match any character *not* in the set. Shorthand classes like `\d`, `\w`, and `\s` are aliases for common sets and work everywhere inside or outside brackets.

CHARACTER CLASS EXAMPLES

```
// — Basic classes —
/[aeiou]/.test("hello")      // true  - any vowel
/[^aeiou]/i.test("aeiou")     // false - no non-vowels
/[a-zA-Z]/.test("Hello")      // true  - any letter
/[0-9a-f]/i.test("1F")        // true  - hex char

// — Ranges —
"abc123".match(/[a-z]+)/      // ["abc"]
"abc123".match(/[0-9]+)/      // ["123"]
"abc123".match(/[a-z0-9]+)/i   // ["abc123"]

// — Shorthands —
"$49.99".match(/\d+/g)        // ["49", "99"]
"foo bar".match(/\S+/g)        // ["foo", "bar"]
"  \t\n".match(/^\s+$/)        // matches - all whitespace

// — Negation —
"hello world".replace(/[^a-z ]/g, "") // "hello world" (strip non-alpha)
"2024-01-15".replace(/^[^d]/g, "")    // "20240115"

// — Combining shorthands inside [ ] —
/[\w-]/.test("my-var")          // true  - word char or hyphen
/[^\s,;]+/g                     // split on whitespace/commas/semicolons
```

KEY RULES

- Inside `[...]` most metacharacters lose their special meaning — `[.++]` matches a literal dot, asterisk, or plus.
- `-` is a range operator only when *between* two characters; place it first or last to match a literal hyphen: `[-az]`.
- `^` is a negation operator only when it is the *first* character inside the brackets.
- `\d` matches `[0-9]`; with Unicode flag it can match non-ASCII digits.
- The `.` metacharacter outside a class matches any char except `\n` (unless `s` flag is set).

NOTES

- `[aeiou]` — explicit set; matches exactly one of those characters
- `[^aeiou]` — negated set; matches any char *not* in the set
- `[a-z0-9]` — two ranges combined; letters or digits
- `\d+` — one or more digits; equivalent to `[0-9]+`
- `\S+` — one or more non-whitespace chars; useful for tokenizing
- `[^d]` and `\D` are identical — both forms are correct
- `[\w-]` — combining a shorthand with a literal hyphen inside a class

SHORTHAND CLASSES

TOKEN	MATCHES	EQUIVALENT CLASS
<code>\d</code>	Any digit	<code>[0-9]</code>
<code>\D</code>	Any non-digit	<code>[^0-9]</code>
<code>\w</code>	Word character	<code>[a-zA-Z0-9_]</code>
<code>\W</code>	Non-word character	<code>[^a-zA-Z0-9_]</code>
<code>\s</code>	Whitespace	<code>[\t\r\n\f\v]</code>
<code>\S</code>	Non-whitespace	<code>[^ \t\r\n\f\v]</code>
<code>.</code>	Any char except <code>\n</code>	<code>[^\n]</code> (without <code>s</code> flag)
<code>\t</code>	Tab	horizontal tab (U+0009)
<code>\n</code>	Newline	line feed (U+000A)
<code>\r</code>	Carriage return	U+000D

UNICODE & POSIX CLASSES

SYNTAX	MATCHES	SUPPORT
<code>\p{L}</code>	Any Unicode letter	Python, JS (u flag), Java
<code>\p{N}</code>	Any Unicode number	Python, JS (u flag)
<code>\p{Lu}</code>	Uppercase letter	Python, Java, .NET
<code>\p{Ll}</code>	Lowercase letter	Python, Java, .NET
<code>\p{Z}</code>	Separator (space etc.)	Python, Java
<code>[:alpha:]</code>	Letters (POSIX)	grep, sed, Ruby
<code>[:digit:]</code>	Digits (POSIX)	grep, sed, Ruby
<code>[:alnum:]</code>	Letters + digits (POSIX)	grep, sed, Ruby
<code>[:space:]</code>	Whitespace (POSIX)	grep, sed, Ruby

COMMON MISTAKES

- Writing `[a-Z]` — the range goes from `a` (97) to `Z` (90) backwards, causing an error in most engines.
- Putting `-` in the middle of a class without escaping: `[a-z0-9-]` is fine only when `-` is last.
- Expecting `[abc]+` to match the string "abc" as a whole — it matches each individual character, not the sequence.
- Using `.` inside `[.]` — the dot loses its special meaning inside brackets and matches only a literal dot.

TIPS

- Prefer `\d`, `\w`, `\s` shorthands over spelled-out ranges — they're shorter and intention-revealing.
- Use `[^n]+` to match a full line without consuming the newline.
- For Unicode-aware matching in JS, add the `u` flag and use `\p{L}` instead of `[a-zA-Z]`.

SUMMARY

- `[...]` matches **one character** from the defined set.
- `[^...]` matches any character **not** in the set.
- Shorthands: `\d` digits · `\w` word chars · `\s` whitespace (and their uppercase negations).
- Ranges use Unicode code points — keep them in ascending order.

WHAT ARE ANCHORS?

Anchors are zero-width assertions — they match a *position* in the string rather than a character. `^` and `$` assert the start and end of the string (or line when multiline mode is on). **Word boundaries** (`\b`) match the transition between a word character and a non-word character, making it possible to match whole words without consuming surrounding characters. Because anchors consume no input, they never appear in the matched text.

ANCHOR USAGE EXAMPLES

```
// — String-level anchors —
/^d{5}$/.test("90210")      // true  — exactly 5 digits
/^d{5}$/.test("902100")     // false — 6 digits
/^Hello/.test("Hello World") // true  — starts with Hello
/World$/.test("Hello World") // true  — ends with World

// — Multiline mode —
const text = "line1\nline2\nline3";
text.match(/^w+/gm)          // ["line1", "line2", "line3"]
text.match(/^w+/)            // ["line1"] — no m flag

// — Word boundaries —
"the cat sat".match(/\bcat\b/) // ["cat"]
"concatenate".match(/\bcat\b/) // null — no word boundary
"price: $100".replace(/\b\d+\b/g, "###") // "price: ###"

// — Python: \A and \Z —
import re
re.match(r'\A\d+\Z', '12345')  # Match
re.match(r'\A\d+\Z', '123\n')  # None — \Z requires true end
re.match(r'^\d+$', '123\n', re.M) # Match — $ allows \n before end
```

KEY RULES

- `^` and `$` match the start/end of the *entire string* by default.
- With the `m` (multiline) flag, `^` / `$` match after/before each `\n`.
- `\b` matches between `\w` and `\W` (or the start/end of the string).
- `\B` is the inverse — it matches where `\b` does *not*.
- Python adds `\A` (absolute start) and `\Z` (absolute end) that ignore the multiline flag.
- Anchors make validation patterns reliable: `^d{5}$` ensures the whole string is 5 digits.

NOTES

- `^d{5}$` — anchoring both ends turns a search into a full-string validation
- `gm` flags together — global finds all matches, multiline lets `^` anchor to each line
- `\bcat\b` — word boundaries prevent matching "cat" inside "catfish" or "concatenate"
- `\b\d+\b` — matches standalone numbers; won't match digits inside identifiers like "abc123"
- `\A` / `\Z` — Python-preferred for validation; immune to multiline flag
- Python `$` allows an optional trailing `\n` — use `\Z` for strict end-of-string

ANCHOR REFERENCE

ANCHOR	MATCHES POSITION...	AFFECTED BY M FLAG?
<code>^</code>	Start of string (or line)	Yes
<code>\$</code>	End of string (or line)	Yes
<code>\A</code>	Absolute start of string	No (Python/Java/.NET)
<code>\Z</code>	Absolute end of string	No (Python/Java/.NET)
<code>\z</code>	Absolute end (strict)	No (Ruby, Java)
<code>\b</code>	Word boundary transition	No
<code>\B</code>	Non-word boundary	No
<code>\G</code>	End of previous match	No (Python, Java, PHP)

BOUNDARY EXAMPLES

PATTERN	STRING	MATCHES
<code>\bcat\b</code>	"the cat sat"	"cat" (whole word)
<code>\bcat\b</code>	"concatenate"	No match
<code>\Bcat\B</code>	"concatenate"	"cat" (mid-word)
<code>^d+\$</code>	"12345"	Match (all digits)
<code>^d+\$</code>	"123a5"	No match
<code>^w+</code>	"hello world"	"hello"
<code>w+\$</code>	"hello world"	"world"
<code>^w+\$</code>	"hello world"	No match (has space)

COMMON MISTAKES

- Using `^` / `$` for per-line matching without the `m` flag — they match the entire string by default.
- Expecting `\b` to work with non-ASCII letters — `\b` is defined by `\w`, which is ASCII-only in most engines.
- Using Python `$` for strict validation — it permits a trailing newline; use `\Z` instead.

TIPS

- Always anchor validation patterns with `^...$` (or `\A...Z` in Python) to prevent partial matches.
- Use `\b` when replacing words to avoid corrupting longer words containing the target.
- Combine `gm` flags in JS when processing multi-line text line by line.

SUMMARY

- Anchors match **positions**, not characters — they consume zero input.
- `^` / `$` — start/end of string; per-line with `m` flag.
- `\A` / `\Z` — absolute string boundaries regardless of flags (Python/Java).
- `\b` / `\B` — word boundary and its inverse for whole-word matching.

HOW QUANTIFIERS WORK

A **quantifier** follows a token (character, class, or group) and specifies *how many times* that token must appear for a match. By default, quantifiers are **greedy** — the engine matches as many characters as possible, then backs off only as needed to satisfy the rest of the pattern. Adding **?** after a quantifier makes it **lazy** (match as few as possible). Some engines support **possessive** quantifiers (**++**, **++**) that never backtrack.

KEY RULES

- Quantifiers apply to the *immediately preceding* token or group.
- *** allows zero matches — it never causes a match to fail on its own.
- Greedy vs lazy only matters when the pattern has content *after* the quantifier.
- {n}** requires exactly n repetitions; **{n,}** means n or more.
- Possessive quantifiers (**++**, **++**) are not supported in JavaScript or Python's **re** module (use **regex** package).

CODE EXPLAINED

- **.*** greedy — expands to end of string, then backtracks to find **>**
- **.*?** lazy — tries shortest match first, expands only if needed
- **a*** — matches empty string; can match at every position
- **\d{3}-\d{4}** — fixed-count quantifiers are ideal for structured data like phone numbers
- **s?** — makes the preceding character optional; common for singular/plural
- Nested quantifiers like **(a+)+** cause catastrophic backtracking — keep quantifier nesting flat

QUANTIFIER EXAMPLES

```
// — Greedy (default) —
'<b>bold</b>'.match(/<.+>/) // ["<b>bold</b>"] — eats everything
'aaa'.match(/a*/)           // ["aaa"]
'bbb'.match(/a*/)           // [""] — zero match succeeds!

// — Lazy —
'<b>bold</b>'.match(/<.+?>/) // ["<b>"] — stops at first >
'<b>bold</b>'.match(/<.+?>/g) // ["<b>", "</b>"]

// — Exact & range —
/\d{4}/.test('2024') // true
/\d{4}/.test('202')  // false
/^ \d{3}-\d{4}$/.test('555-1234') // true — phone segment
/\b\w{5,}\b/g       // all words 5+ chars
```

```
// — Optional with ? —
/https?:\/\/\./.test('http://') // true
/https?:\/\/\./.test('https://') // true
/Jan(?:uary)?/.test('January') // true
/Jan(?:uary)?/.test('Jan')     // true

// — Avoiding catastrophic backtracking —
// BAD: /^(a+)+$/ .test('aaaaaab') — exponential backtrack
// GOOD: /^a+$/ .test('aaaaaab') — no nested quantifier
```

QUANTIFIER REFERENCE		
QUANTIFIER	MEANING	EXAMPLE MATCH
*	0 or more (greedy)	a* → "", "a", "aaa"
+	1 or more (greedy)	a+ → "a", "aaa"
?	0 or 1 (greedy)	colou?r → "color", "colour"
{n}	Exactly n	\d{4} → "2024"
{n,}	n or more	\w{3,} → "cat", "regex"
{n,m}	Between n and m	\d{2,4} → "12", "123", "1234"
*?	0 or more (lazy)	Matches as few as possible
+?	1 or more (lazy)	Matches as few as possible
??	0 or 1 (lazy)	Prefers 0
++	0 or more (possessive)	No backtrack (PCRE/Java)
++	1 or more (possessive)	No backtrack (PCRE/Java)

GREEDY VS LAZY COMPARISON		
PATTERN	INPUT	MATCH
<.+>	"<a>"	"<a>" (greedy)
<.+?>	"<a>"	"<a>" (lazy)
".*"	"x" and "y"	"x" and "y" (greedy — full span)
".*?"	"x" and "y"	"x" (lazy — stops at first "y")
\d{2,4}	"12345"	"1234" (greedy — takes 4)
\d{2,4}?	"12345"	"12" (lazy — takes 2)

COMMON MISTAKES

- Using **.*** when **.*?** is intended — ***** succeeds on an empty string, masking logic errors.
- Forgetting that **?** after a quantifier means *lazy*, not "optional" — **a+?** ≠ **a?**.
- Nesting quantifiers: **(a+)+** causes catastrophic backtracking on non-matching input.
- Writing **{3,5}** with a space: **{ 3, 5 }** — most engines treat the space as a literal, breaking the quantifier.

TIPS

- Prefer **lazy** quantifiers when matching delimited content like HTML tags or quoted strings.
- Use **exact counts** (**{n}**) for structured formats (dates, IDs, codes) — it's self-documenting.
- Watch for **.*** in anchored patterns — if the string can be empty, this always matches.

SUMMARY

- *** / **+** / **?** — zero-or-more, one-or-more, zero-or-one (all greedy by default).
- {n,m}** — explicit repetition bounds for precise control.
- Add **?** after any quantifier to make it **lazy** (shortest match).
- Avoid nested quantifiers to prevent catastrophic backtracking.

GROUPS & BACKREFERENCES

Capturing groups `((...))` serve two purposes: they group tokens so quantifiers apply to the whole unit, and they *capture* the matched text for later use in backreferences or replacements. **Named groups** `((?<name>...))` improve readability by labelling captures.

Non-capturing groups `((?:...))` provide grouping without the overhead of capturing.

Backreferences `(\1, \k<name>)` let you assert that a later part of the string matches something already captured.

KEY RULES

- Groups are numbered left-to-right by opening parenthesis: `((a)(b))` → groups 1, 2, 3.
- Use `(?:...)` when you need grouping but not the capture overhead.
- Named groups: `(?<year>\d{4})` — access via `match.groups.year` in JS or `m.group('year')` in Python.
- Backreference `\1` matches the same text that group 1 captured — not the pattern again.
- Alternation `|` has the lowest precedence — use a group to contain it: `(?:cat|dog)s`.

CODE EXPLAINED

- `m[1]`, `m[2]` — array indices map to capture group numbers (0 = full match)
- `(?<year>...)` — named capture; access via `.groups.year` property on the match
- `(?:...)` — groups the alternation without wasting a capture slot
- `\1` in pattern — backreference; ensures closing tag matches opening tag exactly
- `$1`, `$2` in replacement — inserts captured group by number
- `$<name>` in JS replacement — inserts named capture group by name

GROUPS IN ACTION

```
// — Capturing groups —
const m = "2024-01-15".match(/(\d{4})-(\d{2})-(\d{2})/);
m[0] // "2024-01-15" (full match)
m[1] // "2024" (group 1)
m[2] // "01" (group 2)
m[3] // "15" (group 3)
```

```
// — Named groups (JS) —
const n = "2024-01-15".match(/(?<d{4}>)(?<d{2}>)(?<d{2}>)/);
n.groups.year // "2024"
n.groups.month // "01"
```

```
// — Non-capturing group —
"cats and dogs".match(/(?:cat|dog)s?/g) // ["cats", "dogs"]
```

```
// — Backreferences in pattern —
/(<(\w+)>.*?<\1>/.test("<b>bold</b>") // true — tag must match
/(<(\w+)>.*?<\1>/.test("<b>bold</i>") // false — tags differ
```

```
// — Backreferences in replacement —
"John Smith".replace(/(\w+) (\w+)/, "$2, $1") // "Smith, John"
"2024-01-15".replace(
  /(?<d{4}>)(?<d{2}>)(?<d{2}>)/,
  "$$/$/$"
) // "15/01/2024"
```

GROUP SYNTAX REFERENCE

SYNTAX	TYPE	NOTES
<code>(...)</code>	Capturing group	Numbered from 1; captured text available
<code>(?:...)</code>	Non-capturing group	Groups without capturing; preferred for performance
<code>(?<name>...)</code>	Named capturing group	JS / Python / .NET / Java 7+
<code>(?'name'...)</code>	Named group (alt syntax)	PCRE / .NET only
<code>\1 \2 ...</code>	Numbered backreference	Matches same text as group N
<code>\k<name></code>	Named backreference	JS / Python: <code>(?P=name)</code>
<code>(?P<name>...)</code>	Named group (Python)	Python-specific syntax
<code>(?P=name)</code>	Named backref (Python)	Match same text as named group

ALTERNATION

PATTERN	MATCHES	NOTES
<code>cat dog</code>	"cat" or "dog"	Tries left branch first
<code>(?:cat dog)s</code>	"cats" or "dogs"	Group contains the alternation
<code>gr(?:a e)y</code>	"gray" or "grey"	Efficient mid-word alt
<code>one two three</code>	Any of the three	Left-to-right priority
<code>a ab</code>	"a" in "ab"	Shorter left branch wins in NFA engines
<code>ab a</code>	"ab" in "ab"	Longer branch first = correct greedy alt

COMMON MISTAKES

- Counting groups incorrectly — nested groups add their own number; count by *opening* parentheses left-to-right.
- Using `|` without a group: `cat|dogs` means "cat" OR "dogs", not "cat" OR "dog" + s.
- Expecting `\1` to re-run the group's pattern — it matches the *same captured text*, not a new match of the pattern.

TIPS

- Default to `(?:...)` unless you actually need the capture — it's faster and keeps group numbering clean.
- Named groups make complex patterns self-documenting and immune to renumbering when you add groups.
- Put longer alternation branches *first*: `January|Jan` ensures the full word is tried before the abbreviation.

SUMMARY

- `(...)` — capturing group; numbered by left paren position.
- `(?:...)` — non-capturing group; for grouping only.
- `(?<name>...)` — named capture; access by name in match result.
- `\1` / `$1` — backreference in pattern / replacement.

ZERO-WIDTH LOOKAROUND ASSERTIONS

Lookarounds are zero-width assertions that check what comes *before* or *after* the current position without consuming any characters. A **lookahead** `((?=...))` succeeds if the pattern *can* match ahead; a **negative lookahead** `((?!...))` succeeds if it *cannot*. **Lookbehinds** work the same way but check what precedes the current position. Because they are zero-width, what they check is *never* included in the match result.

KEY RULES

- Lookarounds match a **position** — the checked text is not consumed or captured.
- Lookbehind patterns must be **fixed-width** in most engines (no `*`, `+`, or `?`). Python 3.6+ and .NET allow variable-width lookbehind.
- JavaScript ES2018+ supports lookbehind `((?<=...))` and `((?<!...))`.
- Multiple lookaheads can be chained at the same position: `((?=.*\d)(?=.*[A-Z]))`.
- Lookarounds can be combined with capturing groups — but the lookahead itself does not capture.

LOOKAROUND TYPES

SYNTAX	NAME	SUCCEEDS WHEN...	EXAMPLE
<code>(?=...)</code>	Positive lookahead	Pattern <i>can</i> match ahead	<code>\d(?= dollars)</code> → "5" in "5 dollars"
<code>(?!...)</code>	Negative lookahead	Pattern <i>cannot</i> match ahead	<code>\d(?! dollars)</code> → "9" in "9 euros"
<code>(?<=...)</code>	Positive lookbehind	Pattern <i>can</i> match behind	<code>(?<=\\$)\d+</code> → "100" in "\$100"
<code>(?<!...)</code>	Negative lookbehind	Pattern <i>cannot</i> match behind	<code>(?<!\\$)\d+</code> → "42" in "item 42"

LOOKAROUND EXAMPLES

```
// — Positive lookahead —————
"100px 200em".match(/\d+(?=px)/g)    // ["100"] — numbers before "px"
"file.txt".match(/\w+(?=\.txt)/)      // ["file"] — name without extension

// — Negative lookahead —————
"foo123 bar456".match(/[a-z]+(?!\d)/g) // ["ba"] — letters NOT before digit
"color colour".match(/colou?r(?!\s)/g) // ["color", "colour"]

// — Positive lookbehind —————
"$100 €200 £50".match(/\(?<=\$)\d+/g) // ["100"] — number after $
"width:200px".match(/\(?<=:)\d+/)      // ["200"] — value after colon

// — Negative lookbehind —————
"item 42 $99".match(/\(?<!\$)\d+/g)    // ["42"] — number NOT after $

// — Chained lookaheads (password validation) —————
const strongPwd = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,}$/;
strongPwd.test("Hello1!") // false — too short
strongPwd.test("Hello1!bye") // true

// — Replace without consuming context —————
"1,000,000".replace(/\(?<=\d)(?=(\d{3})+$/g, ",") // "1.000.000"
```

NOTES

- `\d+(?=px)` — matches digits only when "px" follows; "px" is not in the result
- `(?<=\$)\d+` — matches digits only when preceded by "\$"; "\$" is not in the result
- Chained `(?=.*[a-z])(?=.*[A-Z])` — both must be true at the same position; order doesn't matter
- `(?=(\d{3})+)$` — lookahead used as a comma/thousands-separator insertion point
- Lookbehind in JS requires ES2018+ (Chrome 62+, Node 8+, Firefox 78+)
- Variable-length lookbehind not supported in JS — use fixed-width patterns inside `(?<=...)`

COMMON MISTAKES

- Trying to use `+` or `*` inside a lookbehind in JS or most PCRE engines — fixed-width only.
- Expecting the lookahead text to appear in match results — it is never consumed or captured.
- Confusing `(?=abc)` (lookahead) with `(?<=abc)` (lookbehind) — the `<` arrow points backward.

TIPS

- Use **lookbehind** instead of a capturing group when you only need to skip a prefix (e.g. strip the `$` but keep the number).
- Chain multiple lookaheads at position 0 to enforce all-must-pass rules (e.g. password strength).
- Remember: the `<` in `(?<=...)` / `(?<!...)` is a mnemonic — it looks *left* (behind).

SUMMARY

- `(?=...)` lookahead · `(?!...)` negative lookahead — check what follows.
- `(?<=...)` lookbehind · `(?<!...)` negative lookbehind — check what precedes.
- All lookarounds are **zero-width** — checked text is never part of the match.
- Chain lookaheads at the same position for multi-condition validation.

SUBSTITUTION

Substitution replaces matched text with a replacement string. The replacement can reference captured groups via **tokens** like `$1` / `\1` (by number) or `$<name>` / `\g<name>` (by name). A **callback function** (JS) or **lambda** (Python) can be used as the replacement for dynamic transformations. `split()` uses a pattern as a delimiter and optionally captures separator groups into the result array.

KEY RULES

- › In JS, `replace()` without the `g` flag replaces only the **first** match.
- › `replaceAll()` (JS ES2021+) always replaces all matches — requires a string or a `/g` regex.
- › In Python, `re.sub()` replaces all matches by default; use `count=` to limit.
- › Replacement token `$&` (JS) / `\g<0>` (Python) inserts the entire match.
- › A function replacement receives the full match and all group captures as arguments.

CODE EXPLAINED

- `$2, $1` — swaps two captured groups in the replacement string
- `$<name>` — named group reference in JS replacement; cleaner than numbered tokens
- Function as replacer — receives `(match, p1, p2, ..., offset, string)` in JS
- `split(/, \s*/)` — splits on comma plus any trailing whitespace; handles inconsistent spacing
- `re.sub(..., count=2)` — Python's count parameter limits how many replacements are made
- `re.VERBOSE` — allows whitespace and `#` comments inside the pattern for readability

SUBSTITUTION & SPLIT EXAMPLES

```
// — JS: replace with tokens —————
"John Smith".replace(/(\w+) (\w+)/, "$2, $1") // "Smith, John"
"2024-01-15".replace(
  /(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/,
  "$<d>/$<m>/$<y>"
) // "15/01/2024"

// — JS: replace with function —————
"hello world".replace(/b\w/g, c => c.toUpperCase()) // "Hello World"
"a1b2c3".replace(/d/g, n => n * 2) // "a2b4c6"

// — JS: split —————
"one, two, three".split(/, \s*/) // ["one", "two", "three"]
"2024-01-15".split(/[-\/]/) // ["2024", "01", "15"]
```

```
// — Python: re.sub —————
import re
re.sub(r'(\w+) (\w+)', r'\2, \1', 'John Smith') # 'Smith, John'
re.sub(r'\d', lambda m: str(int(m.group())*2), 'a1b2c3') # 'a2b4c6'
re.sub(r'\s+', ' ', 'too many spaces') # 'too many spaces'
re.sub(r'[aeiou]', '*', 'hello', count=2) # 'h*ll*o' — limit 2

# — Verbose mode (Python) —————
pattern = re.compile(r"""
  (\d{4}) # year
  - # separator
  (\d{2}) # month
  - # separator
  (\d{2}) # day
""", re.VERBOSE)
```

REPLACEMENT TOKENS

TOKEN (JS)	TOKEN (PYTHON)	INSERTS...
<code>\$1 \$2 ...</code>	<code>\1 \2 ...</code>	Captured group by number
<code>\$<name></code>	<code>\g<name></code>	Named capture group
<code>\$&</code>	<code>\g<0></code> or <code>\0</code>	Entire match
<code>\$`</code>	—	Text before match (JS only)
<code>\$'</code>	—	Text after match (JS only)
<code>\$\$</code>	—	Literal \$ in replacement string
<code>\U\1</code>	—	Group 1 uppercased (PCRE/sed)
<code>\L\1</code>	—	Group 1 lowercased (PCRE/sed)

FLAG REFERENCE (ALL ENGINES)

FLAG	JS	PYTHON	EFFECT
<code>i</code>	✓	<code>re.I</code> / <code>re.IGNORECASE</code>	Case-insensitive matching
<code>g</code>	✓	default in <code>findall</code>	Global — all matches
<code>m</code>	✓	<code>re.M</code> / <code>re.MULTILINE</code>	<code>^</code> / <code>\$</code> match per line
<code>s</code>	✓	<code>re.S</code> / <code>re.DOTALL</code>	<code>.</code> matches newline
<code>x</code>	✓ (v)	<code>re.X</code> / <code>re.VERBOSE</code>	Ignore whitespace & <code>#</code> comments
<code>u</code>	✓	default in py3	Full Unicode matching
<code>d</code>	✓	—	Indices for substrings (ES2022)

COMMON MISTAKES

- › Forgetting the `g` flag in JS `replace()` — only the first match is replaced without it.
- › Using `$1` tokens in Python's `re.sub()` — Python uses `\1` or `\g<1>` in the replacement string.
- › Writing a literal `$` in a JS replacement string without doubling it — use `$$` for a literal dollar sign.
- › Using `re.VERBOSE` and forgetting to escape spaces that should match — in verbose mode, spaces are ignored.

TIPS

- › Use a **function replacer** in JS for any transformation that can't be expressed with static tokens (case changes, math, lookups).
- › Use `re.VERBOSE` for long, complex patterns — inline comments make them maintainable.
- › Prefer `replaceAll()` over `replace(/pattern/g)` in JS when the pattern is a plain string — it's clearer in intent.

SUMMARY

- › `$1` / `\1` — insert captured group in replacement (JS / Python).
- › Function replacers allow dynamic substitutions beyond static tokens.
- › `split(/pattern/)` — use regex as delimiter for flexible tokenization.
- › `re.VERBOSE` / `x` flag — add whitespace and comments for complex patterns.

SLUG GENERATION

```
function slugify(str) {  
  return str  
    .toLowerCase()  
    .replace(/[^\w\s-]/g, "") // strip punctuation  
    .replace(/[s_]+/g, "-") // spaces/underscores → hyphens  
    .replace(/^+|-+$/g, ""); // trim leading/trailing hyphens  
}  
slugify("Hello, World! 2024") // "hello-world-2024"  
slugify(" __foo & bar__ ") // "foo-bar"
```

USING THESE PATTERNS

- Always anchor validation patterns with `^...$` — without anchors a partial match inside a longer string succeeds.
- Email regex is intentionally simplified — RFC 5322 full compliance requires a 6,000+ char pattern.
- IPv4 uses octet-aware alternation (`25[0-5]|2[0-4]\d|...`) — `\d{1,3}` alone matches 999.
- Password rules use chained lookaheads — each asserts one requirement independently at position 0.
- Test every pattern against both valid *and* invalid examples before shipping.

EMAIL, URL & SLUG

USE CASE	PATTERN	✓ MATCH	✗ NO MATCH
Email	<code>^[\w.+-]+@[\w-]+(?:\.[\w-]+)*\.[a-zA-Z]{2,}\$</code>	user@example.com	bad@.com
URL (http/https)	<code>^https?:\/\/[\w-]+(\.[\w-]+)+(\.[\w-]+~\/?#\[\]@!\$&'()*+,-;= %)*\$</code>	https://example.com/path	ftp://x
Domain	<code>^(?:[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?\.)+[a-zA-Z]{2,}\$</code>	sub.example.co.uk	-bad.com
Slug	<code>^[a-z0-9]+(?:-[a-z0-9]+)*\$</code>	my-page-2024	My Page!

PHONE & DATE

USE CASE	PATTERN	✓ MATCH	✗ NO MATCH
US phone	<code>^(\+1[-. \s]?)?(?(\d{3})?[-. \s]?\d{3}[-. \s]?\d{4})\$</code>	(555) 867-5309	12345
Intl phone	<code>^\+?[1-9]\d{6,14}\$</code>	+447911123456	+0123
ISO date	<code>^\d{4}-(0[1-9] 1[0-2])-(0[1-9] [12]\d 3[01])\$</code>	2024-01-15	2024-13-01
Time (24h)	<code>^([01]\d 2[0-3]):[0-5]\d\$</code>	23:59	25:00

NETWORK & COLORS

USE CASE	PATTERN	✓ MATCH	✗ NO MATCH
IPv4	<code>^(?: (?:25[0-5] 2[0-4]\d [01]?\d\d?)\.) {3} (?:25[0-5] 2[0-4]\d [01]?\d\d?)\$</code>	192.168.1.1	999.0.0.1
MAC address	<code>^([0-9A-Fa-f]{2}[:-]){5}([0-9A-Fa-f]{2})\$</code>	00:1A:2B:3C:4D:5E	GG:HH:II
Hex color	<code>^#([0-9a-fA-F]{3}){1,2}\$</code>	#fff · #1a2b3c	#gg0000
CSS rgb()	<code>^rgb\(\s*(\d{1,3})\s*,\s*(\d{1,3})\s*,\s*(\d{1,3})\s*\)\$</code>	rgb(255, 0, 128)	rgb(256,0,0)

PASSWORD & IDENTITY

USE CASE	PATTERN	✓ MATCH	✗ NO MATCH
Strong password	<code>^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,}\$</code>	PasswOrd!	password
Username	<code>^[a-zA-Z]\w{2,19}\$</code>	jsmith_42	1user · hi
US ZIP	<code>^\d{5}(?:-\d{4})?\$</code>	90210 · 90210-1234	9021
UUID v4	<code>^[0-9a-f]{8}-(?:[0-9a-f]{4}-){3}[0-9a-f]{12}\$</code>	550e8400-e29b-41d4-a716-446655440000	not-a-uuid

COMMON MISTAKES

- Over-engineering email validation — use a simple pattern for UX and confirm ownership via a link instead.
- Using `\d{1,3}` per octet for IPv4 — this matches 999; use the full octet-aware alternation.
- Omitting `^...$` anchors — a pattern without anchors matches anywhere inside the string.
- Logging SSN or credit card match groups — scrub sensitive captures before writing to logs.

TIPS

- Build patterns incrementally on **regex101.com** — paste both ✓ and ✗ cases into the test string panel.
- Use named groups (`(?<year>...)`) for dates and structured IDs — it makes downstream code self-documenting.
- Chain lookaheads at position 0 for password rules — each requirement is independent and easy to add or remove.

SUMMARY

- Anchor all validation patterns with `^...$` to prevent partial matches.
- IPv4 requires octet-aware alternation — `\d{1,3}` alone is insufficient.
- Password strength uses chained lookaheads, one per rule.
- Practical patterns favor correctness for common cases over full spec compliance.

REGEX ACROSS LANGUAGES

The **pattern syntax** is nearly identical across all major languages — the same `\d+`, `(?:...)`, and `(?<name>...)` work everywhere. What differs is the **API**: how you compile, execute, and retrieve results. Understanding these differences lets you translate a pattern from one language to another with minimal effort. The table below maps the same five operations across Python, JavaScript, Java, PHP, and Go.

KEY DIFFERENCES

- Python uses **raw strings** (`r"..."`) to avoid double-escaping backslashes.
- JavaScript regex literals (`/pattern/flags`) don't need quotes; strings passed to `new RegExp()` do require double-escaping.
- Java requires `Pattern.compile()` and double-escaped strings: `"\\d+"`.
- Go's `regexp` package uses RE2 syntax — no lookaheads or backreferences.
- PHP uses PCRE via `preg_*` functions; the pattern must include delimiters: `'/pattern/flags'`.

API COMPARISON — SAME PATTERN, FIVE LANGUAGES

OPERATION	PYTHON	JAVASCRIPT	JAVA	PHP	GO
Import / compile	<code>import re</code> <code>re.compile(r'\d+')</code>	<code>/\d+/ new RegExp("\d+")</code>	<code>import java.util.regex.*; Pattern.compile("\d+")</code>	No import needed	<code>import "regexp" regexp.MustCompile("\d+")</code>
Test (boolean)	<code>bool(re.search(r'\d+', s))</code>	<code>/\d+/.test(s)</code>	<code>Pattern.compile("\d+").matcher(s).find()</code>	<code>preg_match('/\d+/', \$s)</code>	<code>re.MatchString('\d+', s)</code>
First match	<code>re.search(r'\d+', s).group()</code>	<code>s.match(/\d+/)[0]</code>	<code>m.find(); m.group()</code>	<code>preg_match('/\d+/', \$s, \$m); \$m[0]</code>	<code>re.FindString(s)</code>
All matches	<code>re.findall(r'\d+', s)</code>	<code>s.match(/\d+/g)</code>	loop <code>m.find()</code> → <code>m.group()</code>	<code>preg_match_all('/\d+/', \$s, \$m); \$m[0]</code>	<code>re.FindAllString(s, -1)</code>
Replace all	<code>re.sub(r'\d+', 'N', s)</code>	<code>s.replace(/\d+/g, 'N')</code>	<code>p.matcher(s).replaceAll("N")</code>	<code>preg_replace('/\d+/', 'N', \$s)</code>	<code>re.ReplaceAllString(s, "N")</code>
Split	<code>re.split(r',\s+', s)</code>	<code>s.split(/,\s+/)</code>	<code>s.split(",\s+")</code>	<code>preg_split('/,\s+/', \$s)</code>	<code>re.Split(s, -1)</code>
Named groups	<code>m.group('name')</code>	<code>m.groups.name</code>	<code>m.group("name")</code>	<code>\$m['name']</code>	Not supported (RE2)
Flags	<code>re.I, re.M, re.S, re.X</code>	<code>/pat/imsu</code>	<code>Pattern.CASE_INSENSITIVE ...</code>	<code>'/pat/ims'</code>	<code>(?i)</code> inline flags

SIDE-BY-SIDE: EXTRACT YEAR FROM DATE STRING

```
# — Python —
import re
m = re.search(r'(?P<year>)\d{4}-\d{2}-\d{2}', '2024-01-15')
m.group('year')      # '2024'
m.group(1)           # '2024'

# — JavaScript —
const m = '2024-01-15'.match(/(?<year>\d{4})-\d{2}-\d{2}/);
m.groups.year;       // '2024'
m[1];                // '2024'

# — Java —
Pattern p = Pattern.compile("(?<year>\d{4})-\d{2}-\d{2}");
Matcher m = p.matcher("2024-01-15");
if (m.find()) m.group("year"); // "2024"

# — PHP —
preg_match('/(?P<year>\d{4})-\d{2}-\d{2}/', '2024-01-15', $m);
$m['year'];           // "2024"

# — Go (no named groups — use index) —
re := regexp.MustCompile(`(?<year>\d{4})-\d{2}-\d{2}`)
m := re.FindStringSubmatch("2024-01-15")
m[1]                  // "2024"
```

NOTES

- Python — `(?P<name>...)` is the named group syntax; access via `.group('name')`
- JavaScript — `(?<name>...)` syntax; result in `match.groups.name` (ES2018+)
- Java — double-escape all backslashes in string literals: `"\\d+"` not `"\d+"`
- PHP — wrap pattern in delimiters: `'/pattern/flags'`; named groups use `(?P<name>...)`
- Go (RE2) — no lookaheads, no backreferences, no named groups; use submatch indices instead
- Go inline flags — `(?i)` at start of pattern enables case-insensitive matching without a flag argument

COMMON MISTAKES

- Forgetting to double-escape in Java: `"\d+"` is a string escape error; always write `"\\d+"`.
- Omitting delimiters in PHP: `preg_match('\d+', ...)` causes a warning — always use `'/\d+/'`.
- Using lookaheads or backreferences in Go — RE2 intentionally omits them; restructure your pattern or switch to a PCRE library.
- Using `re.match()` in Python thinking it searches the whole string — it only checks from the start.

TIPS

- Compile patterns once and reuse: `re.compile()` in Python, `Pattern.compile()` in Java, `regexp.MustCompile()` in Go.
- In Go, use raw string literals (``...``) to avoid double-escaping — the same benefit as Python's `r"..."`.
- When porting a PCRE pattern to Go, check for lookaheads and backreferences first — those are the most common blockers.

SUMMARY

- Pattern syntax is universal — API wrappers are what differ across languages.
- Python: raw strings · `re.search/findall/sub` · named groups via `(?P<name>...)`.
- Java: double-escape strings · compile once · `Matcher.find()` loop for all matches.
- Go (RE2): no lookaheads or backreferences · use raw backtick strings · submatch index arrays.